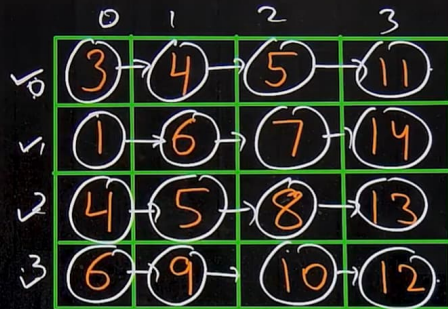


Merge K sorted Array



$K \log n$
 K



$K=4$

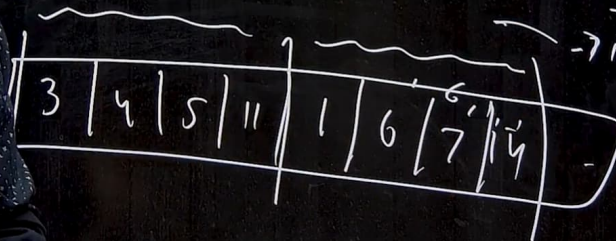
Merge sort

Heap-Sort

$n \log n$

$K^2 \log K^2$

$K^2 + K^2 \log K$ $O(K^2 \log K)$
 $O(K^2 \log K)$ $O(1)$
 $O(K^2)$



$\rightarrow$ Pair K_i $K^2 \log K$ & $O(K^2)$
 $\downarrow$ for $K^2 \log K^2$
 $- K^2 \log K^2$

3	4	5	11	1	6	7	14	4	5	8	13	6	9	10	12
---	---	---	----	---	---	---	----	---	---	---	----	---	---	----	----

20	9	1	3	7	4	2	8
----	---	---	---	---	---	---	---

20	9	1	3
----	---	---	---

7	4	2	8
---	---	---	---

20	9
----	---

1	3
---	---

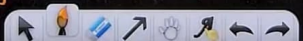
7	4
---	---

2	8
---	---

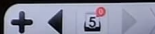
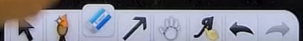
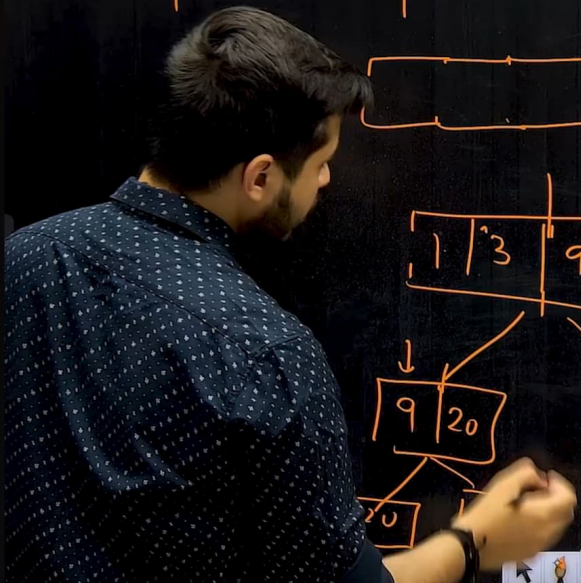
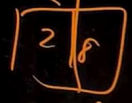
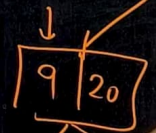
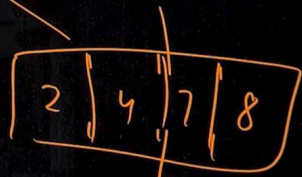
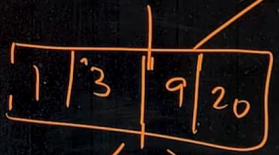
20	9
----	---

7 3

2 8



3	4	5	11	1	6	7	14	4	5	8	13	6	9	10	12
---	---	---	----	---	---	---	----	---	---	---	----	---	---	----	----



3	4	5	11	1	6	7	14	4	5	8	13	6	9	10	12
---	---	---	----	---	---	---	----	---	---	---	----	---	---	----	----

1	2	3	4	7	8	9	20
---	---	---	---	---	---	---	----

1	3	9	20
---	---	---	----

2	4	7	8
---	---	---	---

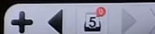
9	20
---	----

1	3
---	---

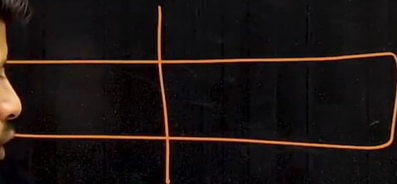
4	7
---	---

2	8
---	---

20	19
----	----



3	4	5	11	1	6	7	14	4	5	8	13	6	9	10	12
---	---	---	----	---	---	---	----	---	---	---	----	---	---	----	----



3	4	5	11
---	---	---	----

1	6	7	14
---	---	---	----

4	5	8	13
---	---	---	----

6	9	10	12
---	---	----	----

3	4	5	11	1	6	7	14	4	5	8	13	6	9	10	12
---	---	---	----	---	---	---	----	---	---	---	----	---	---	----	----

$K=4$ stop-Merge

1	3	4	5	6	7	11	14
---	---	---	---	---	---	----	----

5	6	8	9	10	12	13
---	---	---	---	----	----	----

✓

3	4	5	11
---	---	---	----

✓

4	5	8	13
---	---	---	----

6	9	10	12
---	---	----	----



0	1	2	3	4	5	6	7	8
6	8	10	2	3	9	4	11	14



$K=3$

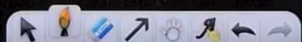
0	1	2	3	4	5	6	7	8
6	8	10	2	3	9	4	11	14

0	1	2	3	4
6	8	10	2	3

9	4	11	14
---	---	----	----

6	8	10
---	---	----

2	3
---	---



K=3

0	1	2	3	4	5	6	7	8
6	8	10	2	3	9	4	11	14

Position = $\frac{3}{2}$
= 1

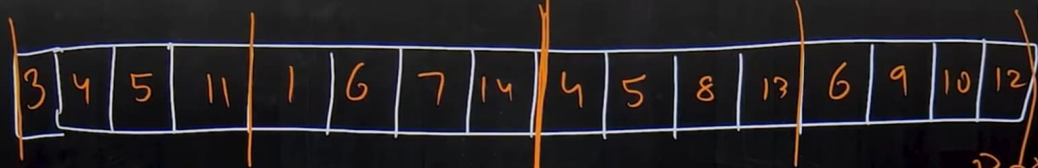
6	8	10
---	---	----

2	3	9	4	11	14
---	---	---	---	----	----

Position = $\frac{2}{2}$
= 1

2	3	9
---	---	---

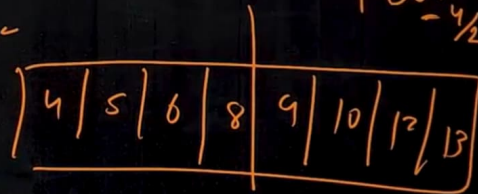
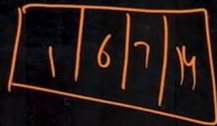
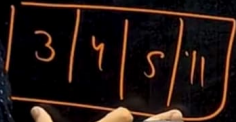
4	11	14
---	----	----



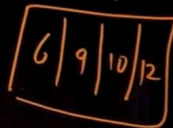
stop - Merge



$\frac{3}{2}$



2



Position
 $-\frac{4}{2}$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
3	4	5	11	1	6	7	14	4	5	8	13	6	9	10	12

stop - Merge

1	3	4	5	6	7	11	14
---	---	---	---	---	---	----	----

K
K/2

K/4
10

$K^2 \log K$

Position
- 1/2

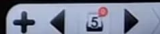
4	5	6	8	9	10	12	13
---	---	---	---	---	----	----	----

3	4	5	11
---	---	---	----

1	6	7	14
---	---	---	----

4	5	8	13
---	---	---	----

6	9	10	12
---	---	----	----



start $\rightarrow$ 0 1 2 3 4 5 6 7 8 $\rightarrow$ end

$k=3$

6	8	10	2	3	9	4	11	14
---	---	----	---	---	---	---	----	----

Position = $\frac{3}{2}$

$= 1$
 $\frac{1}{2}$

6	8	10
---	---	----

2	3	4	9	11	14
---	---	---	---	----	----

Position = $\frac{2}{2}$

$= 1$

$\text{mid} = \frac{\text{start} + \text{end}}{2} \times$

2	3	9
---	---	---

4	11	14
---	----	----

start $\rightarrow$ 0 1 2 3 4 5 6 7 8 $\rightarrow$ end

K=3

6	8	10	2	3	9	4	11	14
---	---	----	---	---	---	---	----	----

Position = $\frac{3}{2}$
= 1

6	8	10
---	---	----

2	3	4	9	11	14
---	---	---	---	----	----

$0 + 3 - 1 = 2$ $1 \times 3 = 3$

mid = start + (Position) / 2 * K - 1

2	3	9
---	---	---

4	11	14
---	----	----

Position = $\frac{2}{2}$
= 1

start $\rightarrow$ 0 1 2 3 4 5 6 7 8 $\rightarrow$ end

$K=3$

6	8	10	2	3	9	4	11	14
---	---	----	---	---	---	---	----	----

start + 3
0 + 3 = 3

6	8	10
---	---	----

mid
3

Position = 3
2
= 1

2	3	4	9	11	14
---	---	---	---	----	----

3/2
0 + 3 - 1

2	3	9
---	---	---

Position = 2
2
= 1

4	11	14
---	----	----

mid = start + (Position) / 2 * K - 1

!!
Double solve

[Problem](#)[Editorial](#)[Submissions](#)[Comments](#)

C++ (g++ 5.4)

Average Time: 45m

[Start Timer](#)

Merge k Sorted Arrays



Medium

Accuracy: 67.25%

Submissions: 84K+

Points: 4

Given K sorted arrays arranged in the form of a matrix of size K*K. The task is to merge them into one sorted array.

Example 1:

Input:

K = 3

arr[][] = {{1,2,3},{4,5,6},{7,8,9}}

Output: 1 2 3 4 5 6 7 8 9

Explanation: Above test case has 3 sorted arrays of size 3, 3, 3

arr[][] = [[1, 2, 3],[4, 5, 6],
[7, 8, 9]]

The merged list will be

[1, 2, 3, 4, 5, 6, 7, 8, 9].

Example 2:

Input:

K = 4

arr[][]={{1,2,3,4},{2,2,3,4},
{5,5,6,6},{7,8,9,9}}

Output:

1 2 2 2 3 3 4 4 5 5 6 6 7 8 9 9

```
15 //User function Template for C++
16
17
18 class Solution
19 {
20     public:
21         //Function to merge k sorted arrays.
22
23
24         vector<int> mergeKArrays(vector<vector<int>> arr, int n)
25         {
26             //code here
27             // merge sort
28             vector<int>ans;
29             for(int i=0;i<K;i++)
30                 for(int j=0;j<K;j++)
31                 {
32                     ans.push_back(arr[i][j]);
33                 }
34
35 }
```

[Custom Input](#)[Compile & Run](#)[Submit](#)



</> Problem

Editorial

🕒 Submissions

💬 Comments

C++ (g++ 5.4)

Average Time: 45m

🕒 Start Timer



Merge k Sorted Arrays



Medium

Accuracy: 67.25%

Submissions: 84K+

Points: 4

Given K sorted arrays arranged in the form of a matrix of size K*K. The task is to merge them into one sorted array.

Example 1:

Input:

K = 3

arr[] = {{1,2,3},{4,5,6},{7,8,9}}

Output: 1 2 3 4 5 6 7 8 9

Explanation: Above test case has 3 sorted arrays of size 3, 3, 3

arr[] = [[1, 2, 3],[4, 5, 6],
[7, 8, 9]]

The merged list will be

[1, 2, 3, 4, 5, 6, 7, 8, 9].

Example 2:

Input:

K = 4

arr[]={{1,2,3,4},{2,2,3,4},
{5,5,6,6},{7,8,9,9}}

Output:

1 2 2 2 3 3 3 4 4 5 5 5 6 6 7 8 9 9

```
18 class Solution
19 {
20     public:
21         //Function to merge k sorted arrays.
22
23
24         vector<int> mergeKArrays(vector<vector<int>> arr, int n)
25         {
26             //code here
27             // merge sort
28             vector<int>ans;
29             for(int i=0;i<K;i++)
30                 for(int j=0;j<K;j++)
31                 {
32                     ans.push_back(arr[i][j]);
33                 }
34
35             mergesort(ans,0,ans.size()-1);
36             return ans;
37         }
38 };
```



Custom Input

Compile & Run

Submit



</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 45m

Start Timer



Merge k Sorted Arrays



Medium

Accuracy: 67.25%

Submissions: 84K+

Points: 4

Given K sorted arrays arranged in the form of a matrix of size K*K. The task is to merge them into one sorted array.

Example 1:

Input:

K = 3

arr[][] = {{1,2,3},{4,5,6},{7,8,9}}

Output: 1 2 3 4 5 6 7 8 9

Explanation: Above test case has 3 sorted arrays of size 3, 3, 3

arr[][] = [[1, 2, 3],[4, 5, 6],
[7, 8, 9]]

The merged list will be

[1, 2, 3, 4, 5, 6, 7, 8, 9].

Example 2:

Input:

K = 4

arr[][]={{1,2,3,4},{2,2,3,4},
{5,5,6,6},{7,8,9,9}}

Output:

1 2 2 2 3 3 4 4 5 5 6 6 7 8 9 9

```
66
67 void mergesort(vector<int>& arr, int start, int end)
68 {
69     if(start==end)
70         return;
71
72     int mid = start+(end-start)/2;
73     // left side
74     mergesort(arr,start,mid);
75     // right side
76     mergesort(arr,mid+1,end);
77     merge(arr,start,mid,end);
78 }
79
80 vector<int> mergeKArrays(vector<vector<int>> arr, in
81 {
82     //code here
83     // merge sort
84     vector<int>ans;
85     for(int i=0;i<K;i++)
86     for(int j=0;j<K;j++)
```



Custom Input

Compile & Run

Submit



</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 45m

Start Timer



Example 1:

Input:

K = 3

arr[] = {{1,2,3},{4,5,6},{7,8,9}}

Output: 1 2 3 4 5 6 7 8 9

Explanation: Above test case has 3 sorted arrays of size 3, 3, 3

arr[] = [[1, 2, 3],[4, 5, 6],

[7, 8, 9]]

The merged list will be

[1, 2, 3, 4, 5, 6, 7, 8, 9].

Example 2:

Input:

K = 4

arr[]={{1,2,3,4},{2,2,3,4},
{5,5,6,6},{7,8,9,9}}

Output:

1 2 2 2 3 3 4 4 5 5 6 6 7 8 9 9

Explanation: Above test case has 4 sorted arrays of size 4, 4, 4, 4

arr[] = [[1, 2, 2, 2], [3, 3, 4, 4],

[5, 5, 6, 6], [7, 8, 9, 9]]

The merged list will be

[1 2 2 2 3 3 4 4 5 5

```

66
67 void mergesort(vector<int>& arr, int start, int end,
68 {
69     if(portion<2)
70     return;
71
72     int mid = start+ (portion/2)*K-1;
73     // left side
74     mergesort(arr,start,mid,portion/2,K);
75     // right side
76     mergesort(arr,mid+1,end,portion-portion/2,K);|
77     merge(arr,start,mid,end);
78 }
79
80 vector<int> mergeKArrays(vector<vector<int>> arr, in
81 {
82     //code here
83     // merge sort
84     vector<int>ans;
85     for(int i=0;i<K;i++)
86     for(int j=0;j<K;j++)
87

```



Custom Input

Compile & Run

Submit



merge k sorted arrays optimized merge sort



The most efficient methods for merging k sorted arrays leverage either a **Min-Heap (Priority Queue)** or a **Divide and Conquer (Merge Sort-like)** approach, both achieving an optimal time complexity of $O(N \log k)$, where N is the total number of elements and k is the number of arrays. GeeksforGeeks +1

1. Min-Heap Approach (Priority Queue)

This approach is highly efficient and the most common solution. A min-heap data structure allows for quick retrieval of the minimum element across all k arrays. Medium +3

• Algorithm:

- **Initialize** a min-heap and insert the first element of each of the k sorted arrays into it, along with its array index and element index.
- **While** the heap is not empty:
 - **Extract** the smallest element from the heap (this is an $O(\log k)$ operation) and add it to the result list.
 - **Insert** the next element from the same array into the heap (also $O(\log k)$) if it exists.

Ask anything





- **While** the heap is not empty:
 - **Extract** the smallest element from the heap (this is an $O(\log k)$ operation) and add it to the result list.
 - **Insert** the next element from the same array into the heap (also $O(\log k)$) if it exists.
- **Return** the final merged sorted array.
- **Time Complexity:** $O(N \log k)$, as each of the N elements is inserted into and extracted from the heap once, and each heap operation takes $O(\log k)$ time.
- **Space Complexity:** $O(N)$ for the output array and $O(k)$ for the heap, resulting in $O(N)$ total space complexity. YouTube +3

2. Divide and Conquer Approach (Optimized Merge Sort)

This method mirrors the recursive nature of a standard merge sort, making it an optimized application of that algorithm. NextWave | CCBP 4.0 C...

- **Algorithm:**
 - **Recursively** split the k arrays into two halves until only single arrays remain (base case).
 - **Merge** the resulting sorted sub-arrays pairwise

Ask anything





merge sort, making it an optimized application of that algorithm. [NxtWave | CCBP 4.0 C...](#)

- **Algorithm:**

- **Recursively** split the k arrays into two halves until only single arrays remain (base case).
- **Merge** the resulting sorted sub-arrays pairwise using a standard two-pointer merge function (an $O(n)$ operation for total n elements in the two combined arrays).
- **Repeat** the merging process up the recursion tree until all arrays are combined into a single sorted array.

- **Time Complexity:** $O(N \log k)$, as there are $\log k$ levels of merging, and each level processes all N elements once.
- **Space Complexity:** $O(N)$ for the output array and $O(\log k)$ for the recursion call stack, resulting in $O(N)$ total space complexity. [NxtWave | CCBP 4....](#) +4

Both methods offer optimal time efficiency for the problem. The min-heap approach might be better suited for external sorting where only a limited number of elements (k) can be in memory at once. The divide and conquer method can also be implemented efficiently and is often expected in interviews. [Reddit +3](#)

Ask anything

